

IS 5203 Lab #2 iPerf3 and Bandwidth

Name: Genevieve McNeil

Date: 03/09/25

[Lab adapted from South Carolina' NTP Lab Series]

This lab briefly introduces iPerf3 and explains how it can be used to measure and test network throughput in a designed network topology. It demonstrates how to invoke both client-side and server-side options from the command line utility.

Lab objectives:

By the end of this lab, students should be able to:

1. Understand throughput and how it differs from bandwidth in network systems.
2. Create iPerf3 tests with various settings on a designed network topology.
3. Understand and analyze iPerf3's test output.
4. Visualize iPerf3's output using a custom plotting script.

Preparation: Install iPerf3 (5 points)

iPerf3 is not pre-installed in Ubuntu systems. After logging into your **GUI** VM, execute:

- `sudo apt update`
- `sudo apt install iperf3`

in the terminal to install the tool.

Preparation: Install plot_iperf.sh (5 points)

To visualize the results you obtained from iPerf3, we need to install a tool "iperf3_plotter" from GitHub.

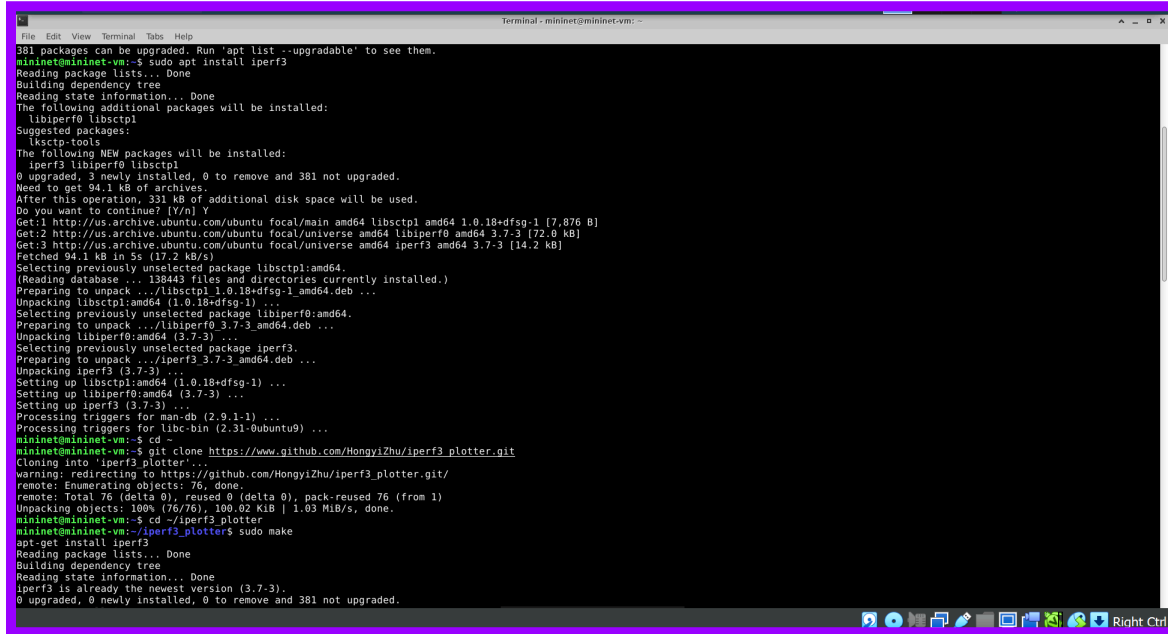
- `$ cd ~`
- `$ git clone https://www.github.com/HongyiZhu/iperf3_plotter.git`
 - Note 1: If the system prompts that git is not installed, please run "sudo apt install git"
 - Note 2: there's a underscore between iperf3 and plotter.git (iperf3_plotter.git)
- `$ cd ~/iperf3_plotter`
- `$ sudo make`

The last command will download required dependencies and compile the tool. Please permit the additional packets to be downloaded to your computer.

Deliverable #1. (1) Execute "iperf3 --version" to demonstrate that the package is properly installed. Please include a screenshot.

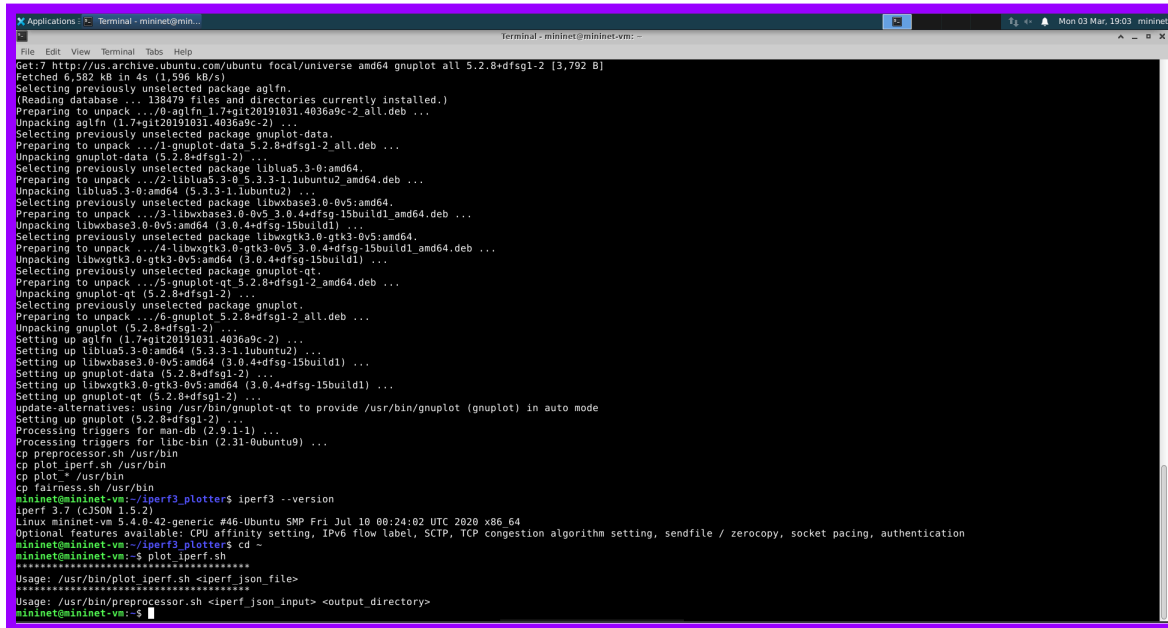
(2) To check if the tool is properly installed, you can navigate to your home folder (\$ `cd ~`) and call `plot_iperf.sh` (\$ `plot_iperf.sh`). If a message prompting correct tool usage shows up, the installation is successful. Please include a screenshot.

Figure: Below is a screenshot of some of the commands I did earlier in the lab.



```
381 packages can be upgraded. Run 'apt list --upgradable' to see them.
mininet@mininet-vm:~$ sudo apt install iperf3
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libiptcpl libiptcpl
Suggested packages:
  libiptcpl-tools
The following NEW packages will be installed:
  iperf3 libiptcpl libiptcpl
0 upgraded, 3 newly installed, 0 to remove and 381 not upgraded.
Need to get 94.1 kB of archives.
After this operation, 331 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://us.archive.ubuntu.com/ubuntu focal/main amd64 libiptcpl amd64 1.0.18+dfsg-1 [7,876 B]
Get:2 http://us.archive.ubuntu.com/ubuntu focal/universe amd64 libiptcpl amd64 1.0.18+dfsg-1 [7,876 B]
Get:3 http://us.archive.ubuntu.com/ubuntu focal/universe amd64 iperf3 amd64 3.7-3 [14.2 kB]
Fetched 94.1 kB in 5s (17.2 kB/s)
Selecting previously unselected package libiptcpl:amd64.
(Reading database ... 138443 files and directories currently installed.)
Preparing to unpack .../libiptcpl_1.0.18+dfsg-1_amd64.deb ...
Unpacking libiptcpl:amd64 (1.0.18+dfsg-1) ...
Selecting previously unselected package libiptcpl:amd64.
Preparing to unpack .../libiptcpl_1.0.18+dfsg-1_amd64.deb ...
Unpacking libiptcpl:amd64 (1.0.18+dfsg-1) ...
Setting up libiptcpl:amd64 (1.0.18+dfsg-1) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for libc-bin (2.31-0ubuntu9) ...
mininet@mininet-vm:~$ cd ~
mininet@mininet-vm:~$ git clone https://www.github.com/HongyiZhu/iperf3_plotter.git
Cloning into 'iperf3_plotter'...
warning: redirecting to https://github.com/HongyiZhu/iperf3_plotter.git/
remote: Enumerating objects: 76, done.
remote: Total 76 (delta 0), reused 0 (delta 0), pack-reused 76 (from 1)
Unpacking objects: 100% (76/76), 100.02 KiB | 1.03 MiB/s, done.
mininet@mininet-vm:~$ cd ~/iperf3_plotter
mininet@mininet-vm:~/iperf3_plotter$ sudo make
apt-get install iperf3
Reading package lists... Done
Building dependency tree
Reading state information... Done
iperf3 is already the newest version (3.7-3).
0 upgraded, 0 newly installed, 0 to remove and 381 not upgraded.
```

Figure: Below is the screenshot containing my results for “iperf3 --version” and “plot_iperf.sh”.



```
Get:7 http://us.archive.ubuntu.com/ubuntu focal/universe amd64 gnuplot all 5.2.8+dfsg1-2 [3,792 B]
Fetched 6,582 kB in 4s (1,596 kB/s)
Selecting previously unselected package gnuplot.
(Reading database ... 138470 files and directories currently installed.)
Preparing to unpack .../0-gnuplot_5.2.8+dfsg1-2_all.deb ...
Unpacking gnuplot (5.2.8+dfsg1-2) ...
Selecting previously unselected package gnuplot-data.
Preparing to unpack .../1-gnuplot-data_5.2.8+dfsg1-2_all.deb ...
Unpacking gnuplot-data (5.2.8+dfsg1-2) ...
Selecting previously unselected package liblua5.3-0:amd64.
Preparing to unpack .../2-liblua5.3-0_5.3.3-1ubuntu2_amd64.deb ...
Unpacking liblua5.3-0:amd64 (5.3.3-1ubuntu2) ...
Selecting previously unselected package libwxbase3.0-0v5:amd64.
Preparing to unpack .../3-libwxbase3.0-0v5_3.0.4+dfsg-1build1_amd64.deb ...
Unpacking libwxbase3.0-0v5:amd64 (3.0.4+dfsg-1build1) ...
Selecting previously unselected package libwxgtk3.0-gtk3-0v5:amd64.
Preparing to unpack .../4-libwxgtk3.0-gtk3-0v5_3.0.4+dfsg-1build1_amd64.deb ...
Unpacking libwxgtk3.0-gtk3-0v5:amd64 (3.0.4+dfsg-1build1) ...
Selecting previously unselected package gnuplot-qt.
Preparing to unpack .../5-gnuplot-qt_5.2.8+dfsg1-2_amd64.deb ...
Unpacking gnuplot-qt (5.2.8+dfsg1-2) ...
Selecting previously unselected package gnuplot.
Preparing to unpack .../6-gnuplot_5.2.8+dfsg1-2_all.deb ...
Unpacking gnuplot (5.2.8+dfsg1-2) ...
Setting up gnuplot (5.2.8+dfsg1-2) ...
Setting up liblua5.3-0:amd64 (5.3.3-1ubuntu2) ...
Setting up libwxbase3.0-0v5:amd64 (3.0.4+dfsg-1build1) ...
Setting up gnuplot-data (5.2.8+dfsg1-2) ...
Setting up libwxgtk3.0-gtk3-0v5:amd64 (3.0.4+dfsg-1build1) ...
Setting up gnuplot-qt (5.2.8+dfsg1-2) ...
update-alternatives: using /usr/bin/gnuplot-qt to provide /usr/bin/gnuplot (gnuplot) in auto mode
Setting up gnuplot (5.2.8+dfsg1-2) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for libc-bin (2.31-0ubuntu9) ...
cp preprocessor.sh /usr/bin
cp plot_iperf.sh /usr/bin
cp plot_* /usr/bin
cp fairness.sh /usr/bin
mininet@mininet-vm:~/iperf3_plotter$ iperf3 --version
iperf 3.7 (cJSON 1.5.2)
Linux mininet-vm 5.4.0-42-generic #46-Ubuntu SMP Fri Jul 10 00:24:02 UTC 2020 x86_64
Optional features available: CPU affinity setting, IPv6 flow label, SCTP, TCP congestion algorithm setting, sendfile / zerocopy, socket pacing, authentication
mininet@mininet-vm:~/iperf3_plotter$ cd ~
mininet@mininet-vm:~$ plot_iperf.sh
Usage: /usr/bin/plot_iperf.sh <iperf_json_file>
Usage: /usr/bin/preprocessor.sh <iperf_json_input> <output_directory>
mininet@mininet-vm:~$
```

Figure: Below is my zoomed in results for “iperf3 --version” and “plot_iperf.sh”

```
cp /usr/bin/fairness.sh /usr/bin
mininet@mininet-vm:~/iperf3_plotters$ iperf3 --version
iperf 3.7 (cJSON 1.5.2)
Linux mininet-vm 5.4.0-42-generic #46-Ubuntu SMP Fri Jul 10 00:24:02 UTC 2020 x86_64
Optional features available: CPU affinity setting, IPv6 flow label, SCTP, TCP congestion algorithm setting, sendfile / zerocopy, socket pacing, authentication
mininet@mininet-vm:~/iperf3_plotters$ cd -
mininet@mininet-vm:~$ plot_iperf.sh
*****
Usage: /usr/bin/plot_iperf.sh <iperf_json_file>
*****
Usage: /usr/bin/preprocessor.sh <iperf_json_input> <output_directory>
mininet@mininet-vm:~$
```

Background:

Bandwidth is a physical property of a transmission media that depends on factors such as the construction and length of wire or fiber. To network engineers, bandwidth is the maximum data rate of a channel, a quantity measured in bits per second (bps). Having a high-bandwidth link does not always guarantee high network performance. In fact, several factors may affect the performance such as latency, packet loss, jitter, and others.

*In the context of a communication session between two end devices along a network path, **throughput** is the rate in bps at which the sending process can deliver bits to the receiving process. Because other sessions will be sharing the bandwidth along the network path, and because these other sessions will recur, the available throughput can fluctuate with time. Note, however, that sometimes the terms throughput and bandwidth are used interchangeably.*

iPerf3 is a real-time network throughput measurement tool. It is an open source, cross-platform client-server application that can be used to measure the throughput between the two end devices. A typical iPerf3 output contains a timestamped report of the amount of data transferred and the throughput measured.

Measuring throughput is particularly useful when experiencing network bandwidth issues such as delay, packet loss, etc. iPerf3 can operate on Transmission Control Protocol (TCP), User Datagram Protocol (UDP), and Stream Control Transmission Protocol (SCTP).

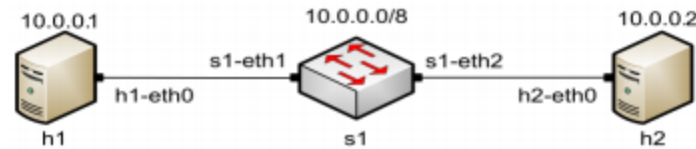
In iPerf3, the user can set **client** and **server** configurations via options and parameters and can create data flows to measure the throughput between the two end hosts in a unidirectional or bidirectional way. iPerf3 outputs a timestamped report of the amount of data transferred and the throughput measured.

Experiment Network Topology

In this Lab, we use the default h1---s1---h2 topology provided by the Mininet environment. We execute:

- \$ sudo mn

to run this network.



Using iPerf3 (35 pts)

Since the initial setup and configuration are done, it is time to start a simple throughput measurement. The user interacts with iPerf3 using the iperf3 command. The basic iperf3 syntax used on both the client and the server is as follows:

iperf3 [-s|-c] [options]

1. Starting client and server

Step 1. We run iPerf3 in server mode on h2. In Mininet CLI, we use

> xterm h2

to call up the terminal on h2.

Step 2. To launch iPerf3 in server mode, run the command # iperf3 -s in host h2's terminal. The parameter -s in the command above indicates that the host is configured as a server.

Deliverable #2. (5 pts) Please show the result of this command.

```

Applications - "Node: h2" - Terminal - mininet@min...
File Edit View Terminal Tabs Help
terminal - mininet@mininet-vm:
Unpacking libxgk3.0-gtk3-0v5:amd64 (3.0.4+dfsg-15build1) ...
Selecting previously unselected package gnuplot-qt.
Preparing to unpack .../5-gnuplot-qt-5.2.8+dfsg1-2_amd64.deb ...
Unpacking gnuplot-qt (5.2.8+dfsg1-2) ...
Selecting previously unselected package gnuplot.
Preparing to unpack .../6-gnuplot-5.2.8+dfsg1-2_all.deb ...
Unpacking gnuplot (5.2.8+dfsg1-2) ...
Setting up aglfn (1.7+git20191031.4036a9c-2) ...
Setting up liblua5.3-0:amd64 (5.3.3-1.1ubuntu2) ...
Setting up libxbase3.0-0v5:amd64 (3.0.4+dfsg-15build1) ...
Setting up gnuplot-data (5.2.8+dfsg1-2) ...
Setting up libxgk3.0-gtk3-0v5:amd64 (3.0.4+dfsg-15build1) ...
Setting up gnuplot-qt (5.2.8+dfsg1-2) ...
update-alternatives: using /usr/bin/gnuplot-qt to provide /usr/bin/gnuplot (gnuplot) in auto mode
Setting up gnuplot (5.2.8+dfsg1-2) ...
Processing triggers for hostmininet-ws/home/mininet# iperf3 -s
Setting: This system does not seem to support IPv6 - trying IPv4
Server listening on 5201
mininet@mininet-vm:~/ip
iperf 3.7 (cJSON 1.5.2)
Linux mininet-vm 5.4.0-
Optional features avail
mininet@mininet-vm:~/ip
mininet@mininet-vm:~$ p
*****
Usage: /usr/bin/plot ip
*****
Usage: /usr/bin/preproc
mininet@mininet-vm:~$ s
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1
*** Starting CLI:
mininet> xterm h2
mininet>
  
```

Which port is the iPerf3 server listening? **It is listening to 5201.**

Step 3. Please call h1's terminal from Mininet CLI (mininet> xterm h1). Now to launch iPerf3 in client mode, run the command # iperf3 -c 10.0.0.2 in host h1's terminal. The parameter -c in command above indicates that host h1 is configured as a client. The parameter 10.0.0.2 is the server's (host h2) IP address.

Once the test is completed, a summary report on both the client and the server is displayed containing the following data:

- ID: identification number of the connection.
- Interval: time interval to periodically report throughput. By default, the time interval is 1 second.
- Transfer: how much data was transferred in each time interval.
- Bitrate: the measured throughput in in each time interval.
- Retr: the number of TCP segments retransmitted in each time interval. This field increases when TCP segments are lost in the network due to congestion or corruption.
- Cwnd: indicates the congestion windows size in each time interval. TCP uses this variable to limit the amount of data the TCP client can send before receiving the acknowledgement of the sent data.

Deliverable #3. (5 pts) Please show the result of this test.

Figure: Results on h1 zoomed out.

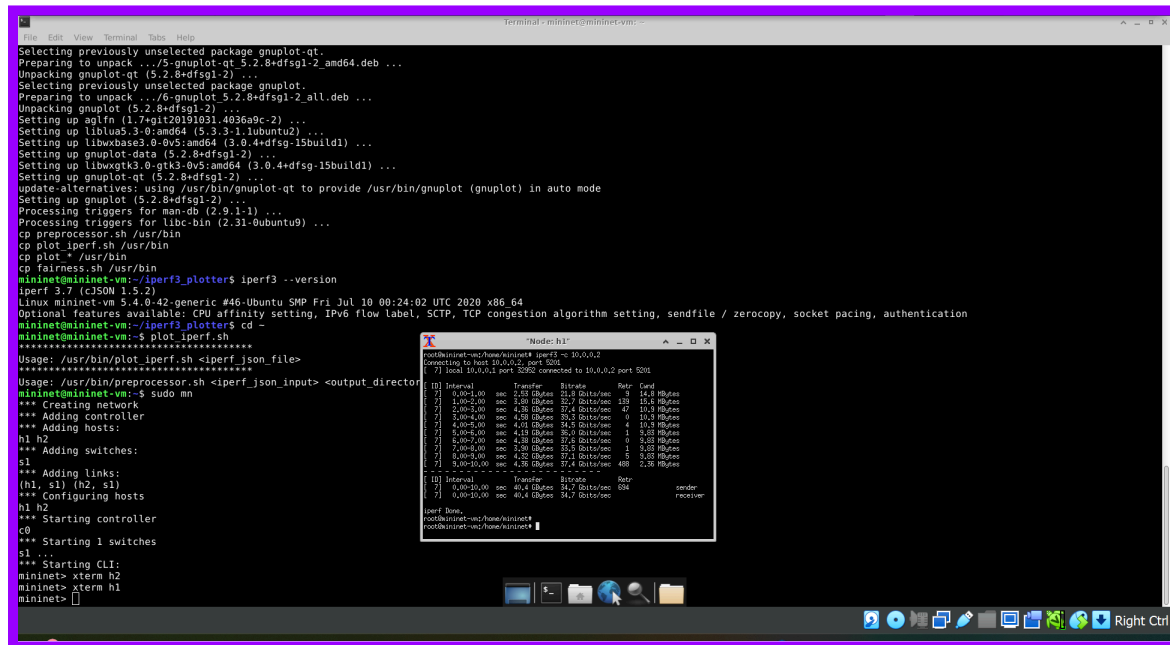


Figure: Results on h1 zoomed in:

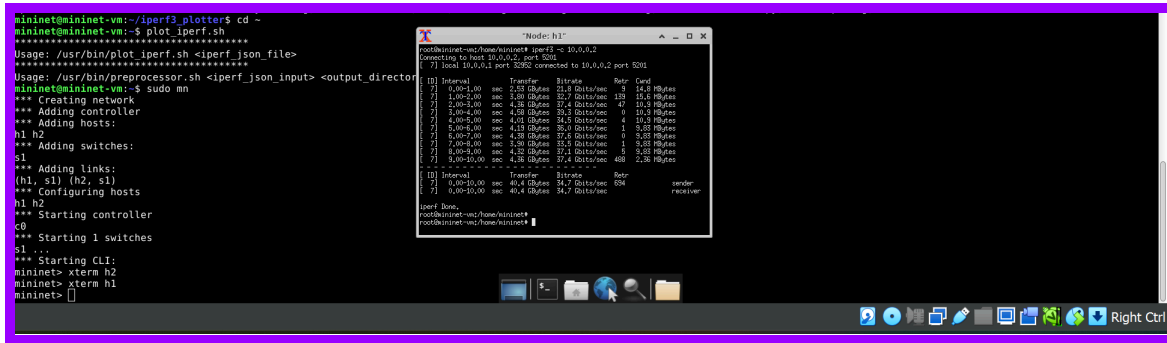
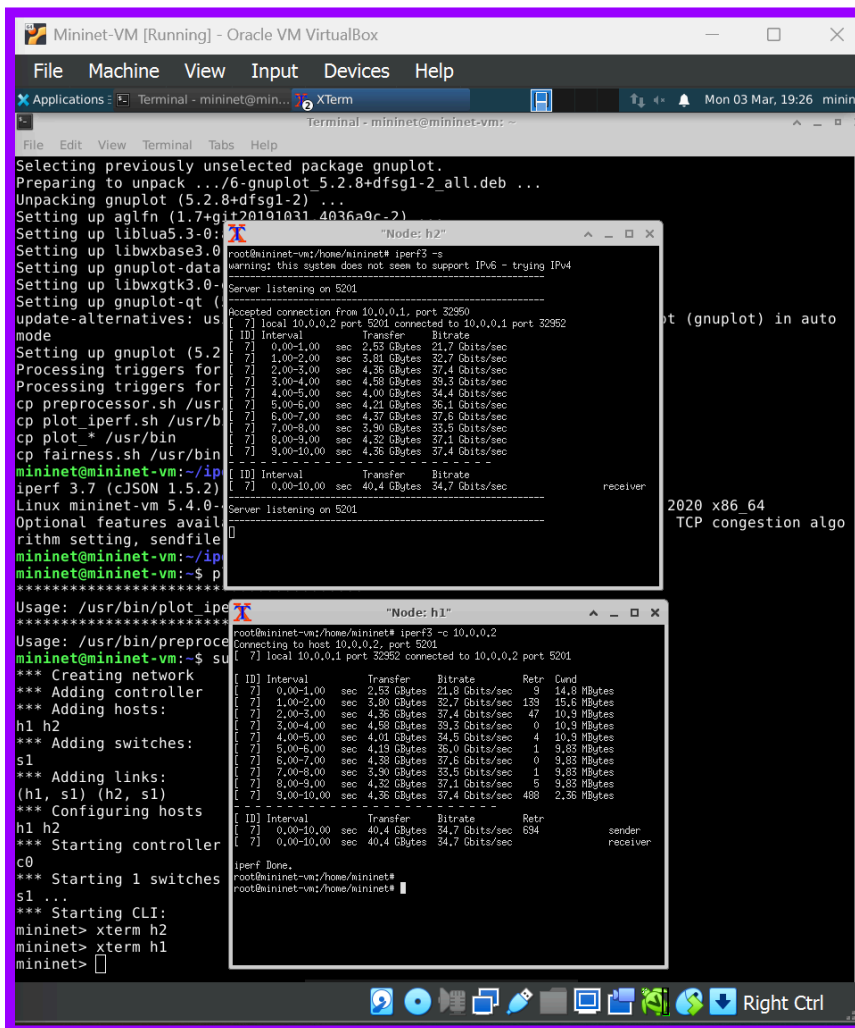


Figure: Below is a photo of both hosts.



What is the total amount of transferred data? What is the averaged bitrate?

Total amount of transferred data: 40.4 GBytes. The average bitrates: 34.7 Gbits/sec.

Step 4. In order to stop the server, press Ctrl+c in host h2's terminal. The user can see the throughput results in the server side too. The summarized data on the server is similar to that of the client side's and must be interpreted in the same way.

2. Setting transmitting time period

Setting the transmission time period is configured solely on the client. To change the default transmission time, apply the following steps:

Step 1. If you have terminated the server, please start the iPerf3 server on h2 again.

Step 2. Start the iPerf3 client on h1 with the -t option followed by the number of seconds (e.g., # iperf3 -c 10.0.0.2 -t 2).

Deliverable #4. (5 pts) Set the transmission period to **5 seconds** and demonstrate the result.

Figure: Below are the results I received after inputting "iperf3 -c 10.0.0.2 -t 5" on h1 (right after restarting the iPerf3 server on h2 again).

```
Mininet-VM [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Applications Terminal - mininet@mininet-vm: ~
Terminal - mininet@mininet-vm: ~
File Edit View Terminal Tabs Help
Selecting previously unselected package gnuplot.
Preparing to unpack .../6-gn...
Unpacking gnuplot (5.2.8+dfsg1-1) ...
Setting up aglfn (1.7+git20180101-1) ...
Setting up liblua5.3-0:amd64 (5.3.3-1) ...
Setting up libwxbase3.0-0v5:amd64 (3.0.2+repack-1) ...
Setting up gnuplot-data (5.2.8+dfsg1-1) ...
Setting up libwxgtk3.0-gtk3-0:amd64 (3.0.2+repack-1) ...
Setting up gnuplot-qt (5.2.8+dfsg1-1) ...
update-alternatives: using /usr/bin/gnuplot-qt to provide /usr/bin/gnuplot in auto mode
Setting up gnuplot (5.2.8+dfsg1-1) ...
Processing triggers for man-db (2.7.6-1) ...
Processing triggers for libc-bin (2.27-3) ...
cp preprocessor.sh /usr/bin
cp plot iperf.sh /usr/bin
cp plot * /usr/bin
cp fairness.sh /usr/bin
mininet@mininet-vm:~/mininet$ iperf3
iperf 3.7 (cJSON 1.5.2)
Linux mininet-vm 5.4.0-42-generic
Optional features available:
rithm setting, sendfile / zero-copy
mininet@mininet-vm:~/mininet$ cd ~
mininet@mininet-vm:~$ plot
*****
Usage: /usr/bin/plot iperf3
*****
Usage: /usr/bin/preprocessor
mininet@mininet-vm:~$ sudo mininet
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> xterm h2
mininet> xterm h1
mininet>
"Node: h1"
[ 7] 4.00-5.00 sec 4.01 GBytes 34.5 Gbits/sec 4 10.9 MBytes
[ 7] 5.00-6.00 sec 4.13 GBytes 36.0 Gbits/sec 1 9.83 MBytes
[ 7] 6.00-7.00 sec 4.38 GBytes 37.6 Gbits/sec 0 9.83 MBytes
[ 7] 7.00-8.00 sec 3.90 GBytes 33.5 Gbits/sec 1 9.83 MBytes
[ 7] 8.00-9.00 sec 4.32 GBytes 37.1 Gbits/sec 5 9.83 MBytes
[ 7] 9.00-10.00 sec 4.36 GBytes 37.4 Gbits/sec 488 2.36 MBytes
-----
[ ID] Interval Transfer Bitrate Retr sender receiver
[ 7] 0.00-1.00 sec 3.04 GBytes 26.1 Gbits/sec 2 2.86 MBytes
[ 7] 1.00-2.00 sec 3.37 GBytes 28.1 Gbits/sec 0 2.57 MBytes
[ 7] 2.00-3.00 sec 3.82 GBytes 32.8 Gbits/sec 55 1.86 MBytes
[ 7] 3.00-4.00 sec 4.08 GBytes 35.1 Gbits/sec 2 1.84 MBytes
[ 7] 4.00-5.00 sec 3.33 GBytes 28.6 Gbits/sec 92 1.51 MBytes
-----
[ ID] Interval Transfer Bitrate Retr sender receiver
[ 7] 0.00-5.00 sec 18.2 GBytes 31.3 Gbits/sec 151
[ 7] 0.00-5.00 sec 18.2 GBytes 31.3 Gbits/sec
-----
iperf Done.
root@mininet-vm:/home/mininet# iperf3 -c 10.0.0.2 -t 5
Connecting to host 10.0.0.2, port 5201
[ 7] local 10.0.0.1 port 32956 connected to 10.0.0.2 port 5201
[ ID] Interval Transfer Bitrate Retr Cwnd
[ 7] 0.00-1.00 sec 3.04 GBytes 26.1 Gbits/sec 2 2.86 MBytes
[ 7] 1.00-2.00 sec 3.37 GBytes 28.1 Gbits/sec 0 2.57 MBytes
[ 7] 2.00-3.00 sec 3.82 GBytes 32.8 Gbits/sec 55 1.86 MBytes
[ 7] 3.00-4.00 sec 4.08 GBytes 35.1 Gbits/sec 2 1.84 MBytes
[ 7] 4.00-5.00 sec 3.33 GBytes 28.6 Gbits/sec 92 1.51 MBytes
-----
[ ID] Interval Transfer Bitrate Retr
[ 7] 0.00-5.00 sec 18.2 GBytes 31.3 Gbits/sec 151
[ 7] 0.00-5.00 sec 18.2 GBytes 31.3 Gbits/sec
-----
iperf Done.
root@mininet-vm:/home/mininet# []
"Node: h2"
[ 7] 8.00-9.00 sec 4.32 GBytes 37.1 Gbits/sec
[ 7] 9.00-10.00 sec 4.36 GBytes 37.7 Gbits/sec
-----
[ ID] Interval Transfer Bitrate
[ 7] 0.00-1.00 sec 3.04 GBytes 26.1 Gbits/sec
[ 7] 1.00-2.00 sec 3.37 GBytes 28.1 Gbits/sec
[ 7] 2.00-3.00 sec 3.82 GBytes 32.8 Gbits/sec
[ 7] 3.00-4.00 sec 4.08 GBytes 35.1 Gbits/sec
[ 7] 4.00-5.00 sec 3.33 GBytes 28.6 Gbits/sec
-----
[ ID] Interval Transfer Bitrate
[ 7] 0.00-5.00 sec 18.2 GBytes 31.3 Gbits/sec
-----
Server listening on 5201
Accepted connection from 10.0.0.1, port 32954
[ 7] local 10.0.0.2 port 5201 connected to 10.0.0.1 port 32956
[ ID] Interval Transfer Bitrate
[ 7] 0.00-1.00 sec 3.04 GBytes 26.1 Gbits/sec
[ 7] 1.00-2.00 sec 3.37 GBytes 28.1 Gbits/sec
[ 7] 2.00-3.00 sec 3.82 GBytes 32.8 Gbits/sec
[ 7] 3.00-4.00 sec 4.08 GBytes 35.1 Gbits/sec
[ 7] 4.00-5.00 sec 3.33 GBytes 28.6 Gbits/sec
-----
[ ID] Interval Transfer Bitrate
[ 7] 0.00-5.00 sec 18.2 GBytes 31.3 Gbits/sec
-----
Server listening on 5201
Ciperf3: interrupt - the server has terminated
root@mininet-vm:/home/mininet# iperf3 -s
warning: this system does not seem to support IPv6 - trying IPv4
Server listening on 5201
```

What is the averaged transmission rate of the client?

The average Bitrate is 31.3 Gbits/sec.

Step 3. In order to stop the server, press Ctrl+c in host h2's terminal. The user can see the throughput results in the server side too.

3. Setting time interval

In this test, the user will configure the client to perform a throughput test with 2-seconds reporting time interval on both the client and the server.

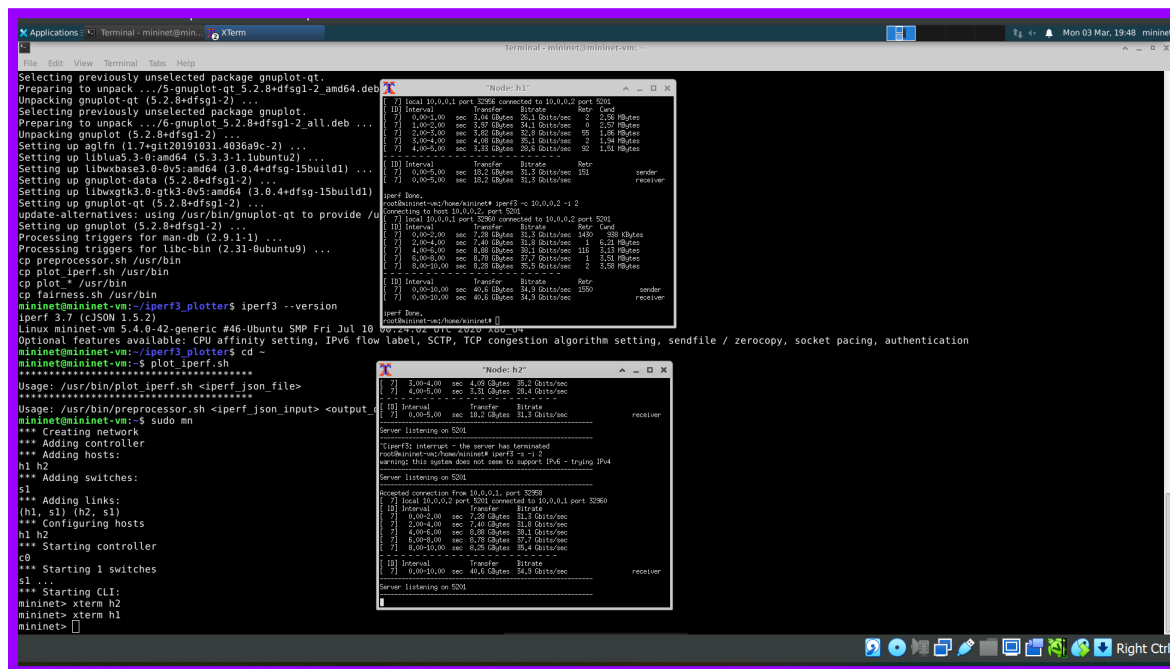
The -i option allows setting the reporting interval time in seconds. In this case the value should be set to 2 seconds on both the client and the server. Note that the -i option can be specified differently on the client and the server. For example, if the -i option is specified with the value 3 on the client only, then the client will be reporting every 3 seconds while the server will be reporting every second (the default -i value).

Step 1. Setting the interval value on the server (host h2's terminal): # iperf3 -s -i 2

Step 2. Setting the interval value on the client (host h1's terminal): # iperf3 -c 10.0.0.2 -i 2

Deliverable #5. (5 pts) Show the execution result on the client.

Figure: Below is a screenshot showing the results of commands in step 1 and step 2 (iperf3 -s -i 2 for host 2 and iperf3 -c 10.0.0.2 -i 2 for host 1).



```
mininet@mininet-vm:~$ sudo apt-get install gnuplot
Selecting previously unselected package gnuplot-qt.
Preparing to unpack ./../6-gnuplot-qt_5.2.8+dfsg1-2_amd64.deb ...
Unpacking gnuplot-qt (5.2.8+dfsg1-2) ...
Selecting previously unselected package gnuplot.
Preparing to unpack ./../6-gnuplot_5.2.8+dfsg1-2_all.deb ...
Unpacking gnuplot (5.2.8+dfsg1-2) ...
Setting up aglfn (1.7+git20191031.4036a9c-2) ...
Setting up liblua5.3-0:amd64 (5.3.3-1ubuntu2) ...
Setting up libxbase3-0:amd64 (3.0.4+dfsg-1build1) ...
Setting up gnuplot (5.2.8+dfsg1-2) ...
Setting up libxgtk3-0-gtk3-0v5:amd64 (3.0.4+dfsg-1build1) ...
Setting up gnuplot-qt (5.2.8+dfsg1-2) ...
update-alternatives: using /usr/bin/gnuplot-qt to provide /usr/bin/gnuplot (5.2.8+dfsg1-2) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for libc-bin (2.31-0ubuntu9) ...
cp preprocessor.sh /usr/bin
cp plot_iperf.sh /usr/bin
cp fairness.sh /usr/bin
mininet@mininet-vm:~$ iperf3 --version
iperf 3.7 (c350N 1.5.2)
Linux mininet-vm 5.4.0-42-generic #46-Ubuntu SMP Fri Jul 10
Optional features available: CPU affinity setting, IPv6 flow
mininet@mininet-vm:~$ iperf3_plotter$ cd -
mininet@mininet-vm:~$ plot_iperf.sh
Usage: /usr/bin/plot_iperf.sh <iperf_json_file>
*****
Usage: /usr/bin/preprocessor.sh <iperf_json_input> <output>
mininet@mininet-vm:~$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> xterm h2
mininet> xterm h1
mininet>

"Node: h1"
[ ] local 10.0.0.1 port 5206 connected to 10.0.0.2 port 5201
[ ] Interval: Transfer: Bitrate: Retr: Cwnd
[ ] 0.00-1.00 sec 3.06 Gbytes 35.1 Gbits/sec 2 2.56 Mbytes
[ ] 1.00-2.00 sec 3.06 Gbytes 34.1 Gbits/sec 1 2.56 Mbytes
[ ] 2.00-3.00 sec 3.06 Gbytes 32.8 Gbits/sec 0 1.98 Mbytes
[ ] 3.00-4.00 sec 4.08 Gbytes 39.1 Gbits/sec 2 1.98 Mbytes
[ ] 4.00-5.00 sec 3.13 Gbytes 29.6 Gbits/sec 0 1.03 Mbytes
[ ] Interval: Transfer: Bitrate: Retr: sender receiver
[ ] 0.00-5.00 sec 18.2 Gbytes 31.3 Gbits/sec 0 0 sender receiver
iperf Done.
root@mininet-vm:/home/mininet# iperf3 -c 10.0.0.2 -i 2
connecting to host 10.0.0.2 port 5201
"Node: h2"
[ ] local 10.0.0.2 port 5201 connected to 10.0.0.1 port 5206
[ ] Interval: Transfer: Bitrate: Retr: Cwnd
[ ] 0.00-1.00 sec 7.40 Gbytes 31.3 Gbits/sec 1 6.21 Mbytes
[ ] 1.00-2.00 sec 7.40 Gbytes 31.3 Gbits/sec 1 6.21 Mbytes
[ ] 2.00-3.00 sec 7.40 Gbytes 31.3 Gbits/sec 1 6.21 Mbytes
[ ] 3.00-4.00 sec 8.28 Gbytes 37.7 Gbits/sec 1 3.51 Mbytes
[ ] 4.00-5.00 sec 8.28 Gbytes 37.3 Gbits/sec 2 3.51 Mbytes
[ ] Interval: Transfer: Bitrate: Retr: sender receiver
[ ] 0.00-10.00 sec 40.8 Gbytes 34.3 Gbits/sec 350 0 sender receiver
iperf Done.
root@mininet-vm:/home/mininet#
```

How many intervals were reported? How long is each interval?

5 intervals were reported and they were 2 seconds each.

Step 3. In order to stop the server, press Ctrl+c in host h2's terminal. The user can see the throughput results in the server side too.

4. Changing the number of bytes to transmit

In this test, the client is configured to send a specific amount of data by setting the number of bytes to transmit. By default, iPerf3 performs the throughput measurement for 10 seconds. However, with this configuration, the client will keep sending packets until all the bytes specified by the user were sent.

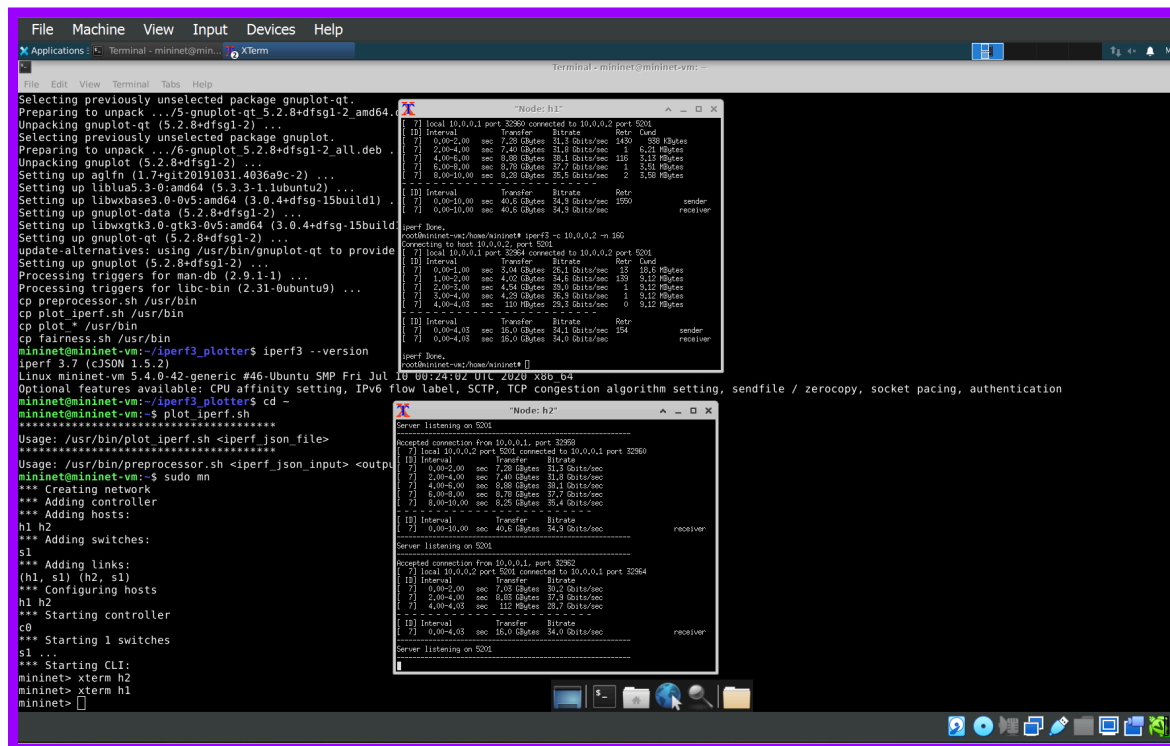
Step 1. Start the iPerf3 on h2.

Step 2. This configuration is only set on the client (host h1's terminal) using the -n option as follows: # iperf3 -c 10.0.0.2 -n 16G

The -n option in the above command indicates the amount of data to transmit: 16 Gbytes. The user can specify other scale values, for example, 16M is used to send 16 Mbytes.

Deliverable #6. (5 pts) Show the execution result on the client.

Figure: Below is a screenshot of my results for inputting the command: `iperf3 -c 10.0.0.2 -n 16G` on host 1.



How long does it take for the client to send 16GB data? When does the server receive the last bit of data?

It takes the client in total: 4.03 seconds. It noted that the last bit of data was received at 4.03 seconds.

Step 3. In order to stop the server, press Ctrl+c in host h2's terminal. The user can see the throughput results in the server side too.

5. Specifying the transport-layer protocol

So far, the throughput measurements were conducted on the TCP protocol, which is the default configuration protocol. In order to change the protocol to UDP, the user must invoke the option -u on the client side. Similarly, the option --sctp is used for the SCTP protocol. iPerf3 automatically detects the transport-layer protocol on the server side.

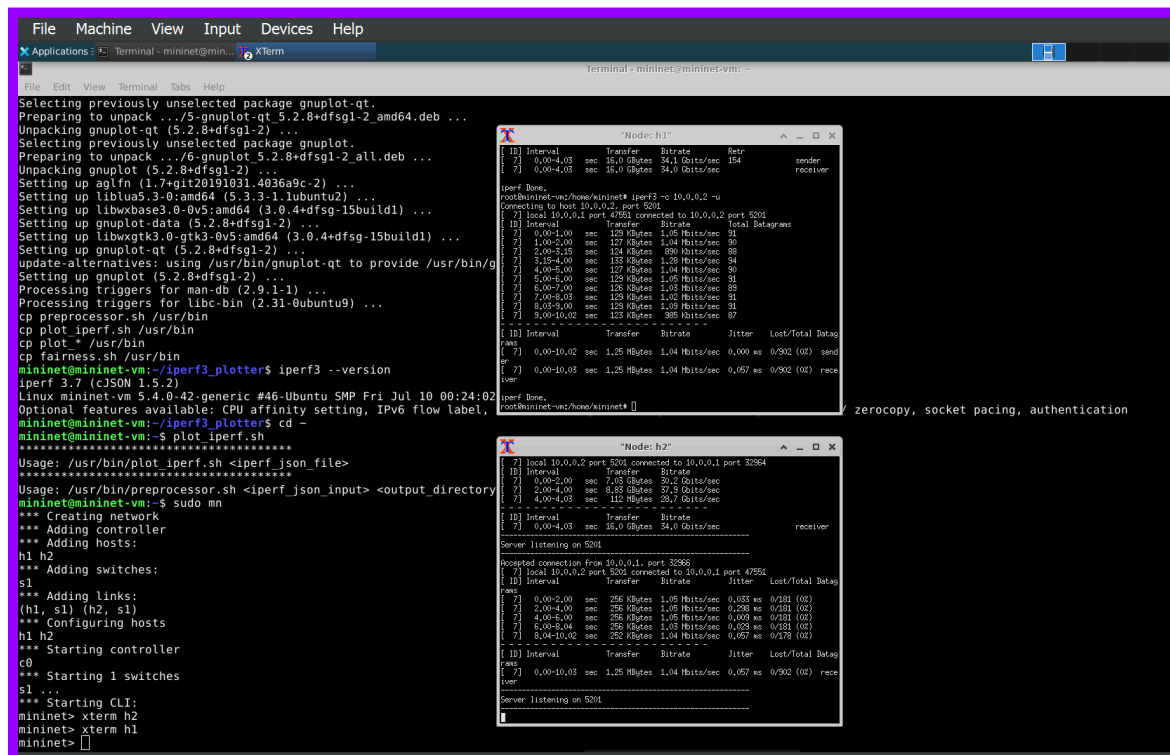
Step 1. Start the iPerf3 on h2.

Step 2. Specify UDP as the transport-layer protocol using the -u option as follows: # iperf3 -c 10.0.0.2 -u. Once the test is completed, it will show the following summarized data:

- ID, Interval, Transfer, Bitrate: Same as TCP.
- Jitter: the difference in packet delay.
- Lost/Total: indicates the number of lost datagrams over the total number sent to the server (and percentage).

Deliverable #7. (5 pts) Show the execution result on the client.

Figure: Below is a screenshot of inputting the command “iperf3 -c 10.0.0.2 -u” on host1.



```
File Machine View Input Devices Help
Applications: Terminal - mininet@mininet-vm: ~
File Edit View Terminal Tabs Help
Selecting previously unselected package gnuplot-qt.
Preparing to unpack .../5-gnuplot-qt_5.2.8+dfsg1-2_amd64.deb ...
Unpacking gnuplot-qt (5.2.8+dfsg1-2) ...
Selecting previously unselected package gnuplot.
Preparing to unpack .../6-gnuplot_5.2.8+dfsg1-2_all.deb ...
Unpacking gnuplot (5.2.8+dfsg1-2) ...
Setting up aglfn (1.7+git20191031.4036a9c-2) ...
Setting up liblua5.3-0:amd64 (5.3.3-1.1ubuntu2) ...
Setting up libxbase64-0-0v5:amd64 (3.0.4+dfsg-15build1) ...
Setting up gnuplot-data (5.2.8+dfsg1-2) ...
Setting up libxgk3.0-gtk3-0v5:amd64 (3.0.4+dfsg-15build1) ...
Setting up gnuplot-qt (5.2.8+dfsg1-2) ...
update-alternatives: using /usr/bin/gnuplot-qt to provide /usr/bin/g
Setting up gnuplot (5.2.8+dfsg1-2) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for libc-bin (2.31-0ubuntu9) ...
cp preprocessor.sh /usr/bin
cp iperf.sh /usr/bin
cp plot * /usr/bin
cp fairness.sh /usr/bin
mininet@mininet-vm:~/iperf3_plotter$ iperf3 --version
iperf 3.7 (c350N 1.5.2)
Linux mininet-vm 5.4.0-42-generic #46-Ubuntu SMP Fri Jul 10 00:24:02
Optional features available: CPU affinity setting, IPv6 flow label,
mininet@mininet-vm:~/iperf3_plotter$ cd -
mininet@mininet-vm:~$ plot_iperf.sh
*****
Usage: /usr/bin/plot_iperf.sh <iperf_json_file>
*****
Usage: /usr/bin/preprocessor.sh <iperf_json_input> <output_directory>
mininet@mininet-vm:~$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
mininet> xterm h2
mininet> xterm h1
mininet> [ ]
```

Is there any jitter during the transmission?

Yes there is. In the figure above, there is a column called "Jitter" with a list of 5 jitters: 0.033 ms, 0.298 ms, 0.009 ms, 0.029 ms, and 0.057 ms.

Step 3. In order to stop the server, press Ctrl+c in host h2's terminal. The user can see the throughput results in the server side too.

6. Export results to JSON file

So far, the throughput measurements were conducted on the TCP protocol, which is the default configuration protocol. In order to change the protocol to UDP, the user must invoke the option -u on the client side. Similarly, the option --sctp is used for the SCTP protocol. iPerf3 automatically detects the transport-layer protocol on the server side.

Step 1. Start the iPerf3 server on h2.

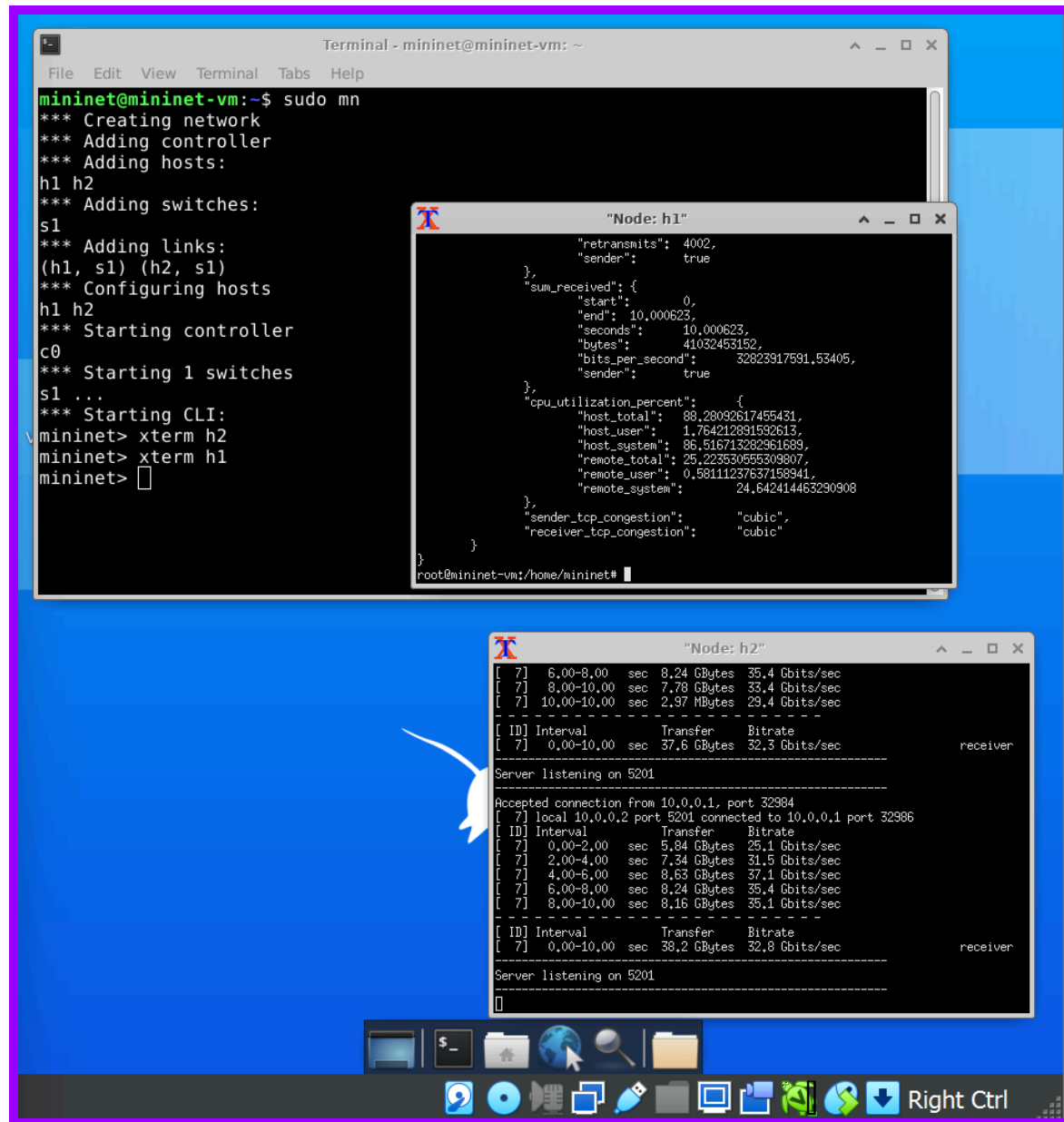
Step 2. Start the iPerf3 client on host h1. Specify the -J option to display the output in JSON format: # iperf3 -c 10.0.0.2 -J

It is often useful to export the output to a file that can be parsed later by other programs. This can be done by redirecting the standard output to a file using the redirection operator in Linux >:
iperf3 -c 10.0.0.2 -J > results.json

- Note: Please wait until the command finishes execution (i.e., the next # prompt shows up).

After creating the JSON file, the ls command is used to verify that the file is created. The cat command can be used to display the file's contents.

Deliverable #8. (5 pts) You want to change directory (i.e., cd) to the current work path (where you launched sudo mn, likely to be your home folder "~" or "/home/mininet") to find the exported result file. Display your test result stored in the json file using cat <result filename> (e.g., if your result filename is results.json, you will run "cat results.json" in the folder that contains the result file).



Step 3. In order to stop the server, press Ctrl+c in host h2's terminal. The user can see the throughput results in the server side too.

Plotting iPerf3 results (5 pts)

In the previous section, iPerf3's result was exported to a JSON file to be processed by other applications. A script called **plot_iperf.sh** is installed during the preparation stage. It accepts a JSON file as input and generates PDF files plotting several variables produced by iPerf3.

This plotting script generates PDF files for the following fields: congestion window (cwnd.pdf), retransmits (retransmits.pdf), Round-Trip Time (RTT.pdf), Round-Trip Time variance

(RTT_Var.pdf), throughput (throughput.pdf), maximum transmission unit (MTU.pdf), bytes transferred (bytes.pdf). The plotting script also generates a CSV file (1.dat) which can be used by other applications. These files are stored in a directory results created in the same directory where the script was executed.

Step 1. Evoke a Linux terminal. Execute the script to plot the results. Please navigate to the “results” folder that has the result JSON file.

```
$ plot_iperf.sh results.json
```

```
$ cd results
```

Step 2. Install Evince (\$ sudo apt install evince) to browse the generated PDF files.

Deliverable #9 (5 pts) Please show the content (the diagram) of bytes.pdf and cwnd.pdf (you can paste a screenshot of the pdf file content below). What can you interpret from the diagrams?

Figure: Below is a screenshot of Bytes.pdf zoomed in:

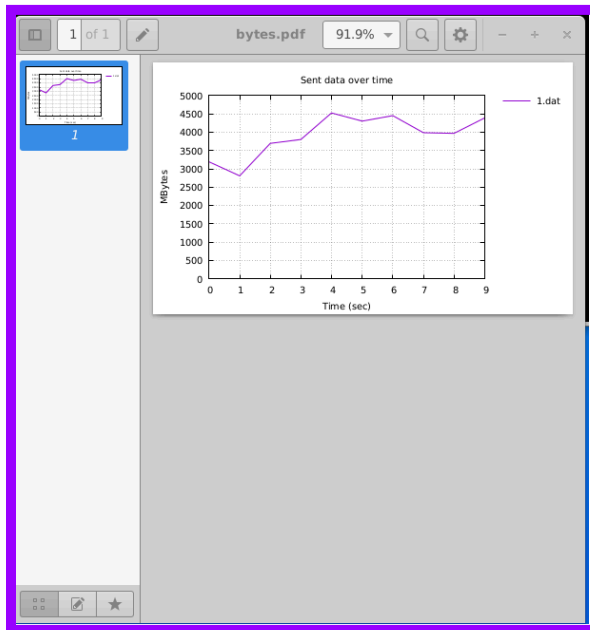
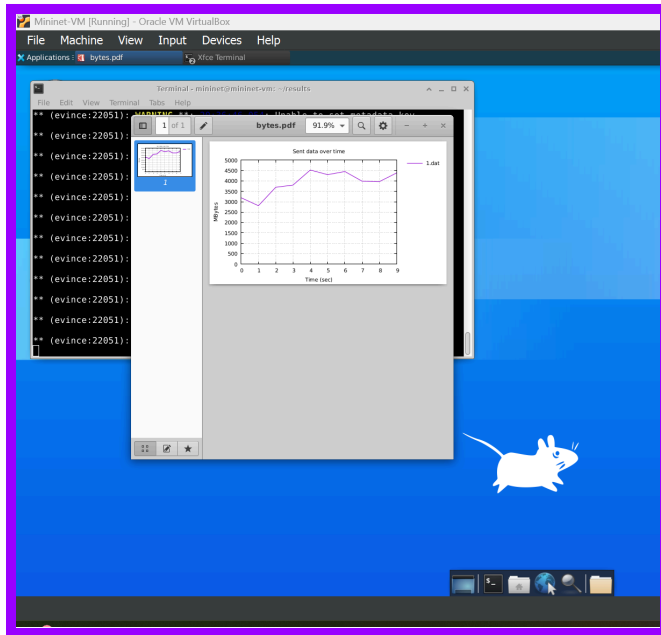


Figure: Below is a screenshot of Bytes.pdf zoomed out:



Interpretation of Bytes.pdf:

This diagram above displays the amount of data that was sent over a specific amount of time. X-axis equals the time (seconds), while y equals the amount of bytes sent. One thing I would like to note is that there are fluctuations in the graph. To me that shows that there may be packet loss or even congestion occurring. Also, the data transfer rate essentially increases over time. This is because there is a trend in the graph that goes upward.

Figure: On the next page is a screenshot for cwnd.pdf zoomed out:

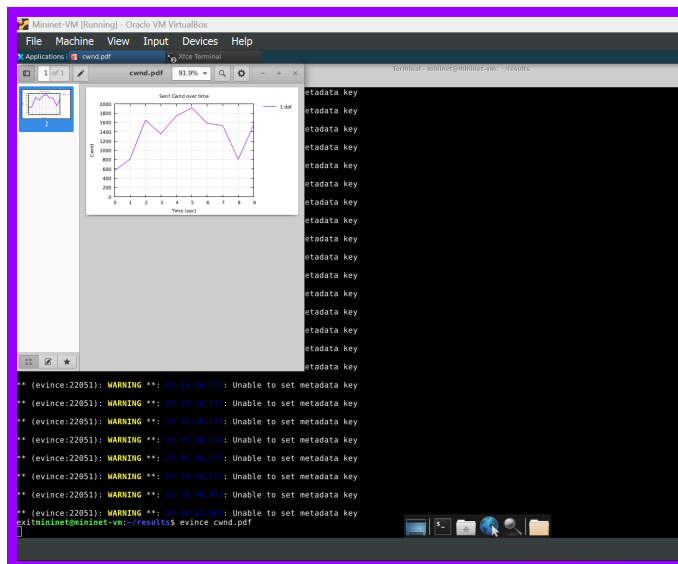
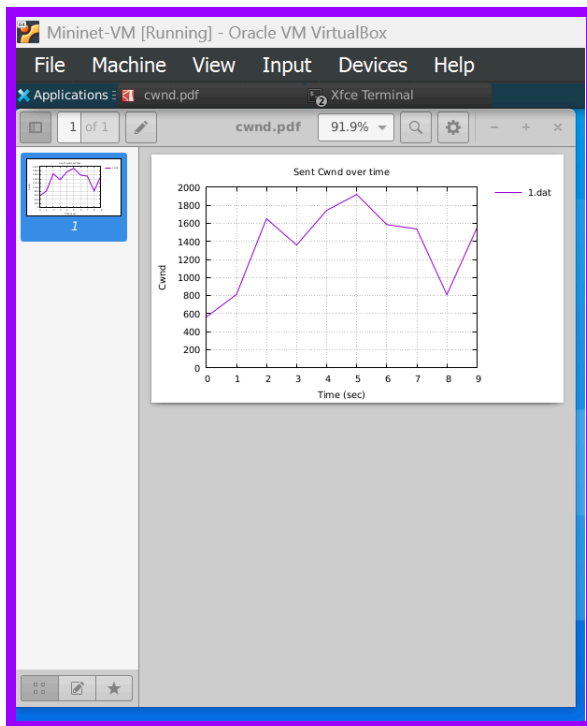


Figure: Below is a screenshot for cwnd.pdf zoomed in:



Interpretation of cwnd.pdf:

The y-axis is the size of the cwnd (congestion window) whilst the x-axis is the duration of time (seconds). In the beginning of the graph, there is a lot of data sent. Then at about 2 seconds, there is a fluctuation, which could be due to congestion or packet loss. Then the graph starts to stabilize once again. There is a steep drop later on in the graph, which also could be caused by packet loss or congestion. To me, this all means that the sender is trying to avoid congestion whilst sending as much data as possible. It also shows that when there is packet loss or congestion, the network attempts to stabilize.